

AFRILANG Stdlib

std/c/tsma

Bibliothèque AFRILANG `std/c/tsma` (niveau complex, catégorie complex). Elle expose 6 fonction(s) : sma, wma, dema, cros

```
`import "std/c/tsma" · `use tsma`
```

- `sma(v list number, win number) ? list number`
- `wma(v list number, win number) ? list number`
- `dema(v list number, win number) ? list number`
- `crossover(v list number, fast number, slow number) ? list number`
- `signal(v list number, fast number, slow number) ? number`
- `trend(v list number, win number) ? number`

Category: complex | Tier: complex | Functions: 6

FRANCAIS

1. Vue d'ensemble

Bibliothèque AFRILANG `std/c/tsma` (niveau complex, catégorie complex). Elle expose 6 fonction(s) : sma, wma, dema, cros

```
`import "std/c/tsma" . `use tsma`  
- `sma(v list number, win number) ? list number`  
- `wma(v list number, win number) ? list number`  
- `dema(v list number, win number) ? list number`  
- `crossover(v list number, fast number, slow number) ? list number`  
- `signal(v list number, fast number, slow number) ? number`  
- `trend(v list number, win number) ? number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/tsma"  
use tsma
```

Niveau : complex · Catégorie : Modules complex (c/)

Domaines avancés ? graphes, NLP, réseaux complexes.

3. Référence API

```
? sma(v list number, win number) ? list  
? wma(v list number, win number) ? list  
? dema(v list number, win number) ? list  
? crossover(v list number, fast number, slow number) ? list  
? signal(v list number, fast number, slow number) ? number  
? trend(v list number, win number) ? number
```

4. Exemples d'utilisation

```
import "std/c/tsma"  
use tsma  
  
create result = sma(/* args */)   
say result  
  
create result = wma(/* args */)   
say result  
  
create result = dema(/* args */)   
say result  
  
create result = crossover(/* args */)   
say result  
  
create result = signal(/* args */)   
say result  
  
create result = trend(/* args */)   
say result
```

5. Bonnes pratiques

```
? Préférez `import "std/c/tsma"` en tête de fichier.  
? Lancez `afrilang check` avant de livrer.  
? Combinez avec les modules stdlib de la même catégorie.  
? Voir /stdlib/ pour le source live et les démos playground.  
? Signalez les problèmes sur GitHub si une export manque.
```

6. Annexe ? source

```
module tsma  
  
function sma(v list number, win number) returns list number  
end
```

```
function wma(v list number, win number) returns list number
end

function dema(v list number, win number) returns list number
end

function crossover(v list number, fast number, slow number) returns list number
end

function signal(v list number, fast number, slow number) returns number
end

function trend(v list number, win number) returns number
end
end
```

ENGLISH

1. Overview

AFRILANG library `std/c/tsma` (tier complex, category complex). Exports 6 function(s): sma, wma, dema, crossover, signal

```
`import "std/c/tsma" . `use tsma`  
- `sma(v list number, win number) ? list number`  
- `wma(v list number, win number) ? list number`  
- `dema(v list number, win number) ? list number`  
- `crossover(v list number, fast number, slow number) ? list number`  
- `signal(v list number, fast number, slow number) ? number`  
- `trend(v list number, win number) ? number`
```

2. Installation & import

Add the module to your program:

```
import "std/c/tsma"  
use tsma
```

Tier: complex · Category: Complex modules (c/)
Advanced domains ? graphs, NLP, complex networks.

3. API reference

```
? sma(v list number, win number) ? list  
? wma(v list number, win number) ? list  
? dema(v list number, win number) ? list  
? crossover(v list number, fast number, slow number) ? list  
? signal(v list number, fast number, slow number) ? number  
? trend(v list number, win number) ? number
```

4. Usage examples

```
import "std/c/tsma"  
use tsma  
  
create result = sma(/* args */)   
say result  
  
create result = wma(/* args */)   
say result  
  
create result = dema(/* args */)   
say result  
  
create result = crossover(/* args */)   
say result  
  
create result = signal(/* args */)   
say result  
  
create result = trend(/* args */)   
say result
```

5. Best practices

```
? Prefer `import "std/c/tsma"` at the top of your file.  
? Run `afirang check` before shipping.  
? Combine with related stdlib modules from the same category.  
? See /stdlib/ for the live source and playground demos.  
? Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix ? source

```
module tsma  
  
function sma(v list number, win number) returns list number  
end
```

```
function wma(v list number, win number) returns list number
end

function dema(v list number, win number) returns list number
end

function crossover(v list number, fast number, slow number) returns list number
end

function signal(v list number, fast number, slow number) returns number
end

function trend(v list number, win number) returns number
end
end
```